

In [7]: `import numpy as np`

```
data = np.load('octmnist.npz')
print("Keys in octmnist.npz: ", data.files)

for k in data.files:
    arr = data[k]
    print(f"{k:>12}: shape = {arr.shape}")

train_images = data["train_images"]
train_labels = data["train_labels"].ravel()

val_images = data["val_images"]
val_labels = data["val_labels"].ravel()

test_images = data["test_images"]
test_labels = data["test_labels"].ravel()
```

```
Keys in octmnist.npz: ['train_images', 'val_images', 'test_images', 'train_labels',
'val_labels', 'test_labels']
train_images: shape = (97477, 28, 28)
val_images: shape = (10832, 28, 28)
test_images: shape = (1000, 28, 28)
train_labels: shape = (97477, 1)
val_labels: shape = (10832, 1)
test_labels: shape = (1000, 1)
```

Loading Necessary Data. Taken Straight out of a Previous Notebook

```
In [8]: from models import stratified_subset
from sklearn.utils.class_weight import compute_sample_weight
import tensorflow as tf
from pathlib import Path
from time import strftime

x_tr, y_tr = stratified_subset(train_images, train_labels, 22000)
x_va, y_va = stratified_subset(val_images, val_labels, 2000)

sample_weights = compute_sample_weight(class_weight='balanced', y=y_tr)

early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=10,
                                                    restore_best_weights=True)

def get_run_logdir(root_logdir='my_logs'):
    return Path(root_logdir)/strftime("run_%Y_%m_%d_%H_%M_%S")

run_logdir = get_run_logdir()

tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir,
                                                profile_batch=(100, 200))
```

Early Prep. Also Taken Straight out of a Previous Notebook.

```
In [9]: x_tr_flat = x_tr.reshape(x_tr.shape[0], -1)
```

Model Tuning

```
In [10]: from scipy.stats import loguniform, randint
from sklearn.model_selection import RandomizedSearchCV
from models import median, avg_2x2_pool, flatten_data

x_tr_stacking = flatten_data(avg_2x2_pool(median(x_tr))).astype(np.float32)
```

Model 1: Preprocessed Stacking

```
In [11]: from models import build_preprocessed_stacking, build_preprocessed_stacking_tunable

stacking_param_dist = {
    "svc__kernel__n_components": [300, 500, 700],
    "svc__svc__C": loguniform(0.1, 10),
    "final_estimator__C": loguniform(0.1, 10),
}

stacking_search = RandomizedSearchCV(
    estimator=build_preprocessed_stacking_tunable(),
    param_distributions=stacking_param_dist,
    n_iter=10,
    cv=2,
    scoring="accuracy",
    n_jobs=-1,
    verbose=2,
    random_state=42,
    refit=True,
)

stacking_search.fit(x_tr_stacking, y_tr)

print("Best stacking params:", stacking_search.best_params_)
print(f"Best CV accuracy: {stacking_search.best_score_:.4f}")
stacking_clf_tuned = build_preprocessed_stacking()
stacking_clf_tuned.set_params(**stacking_search.best_params_)
```

Fitting 2 folds for each of 10 candidates, totalling 20 fits

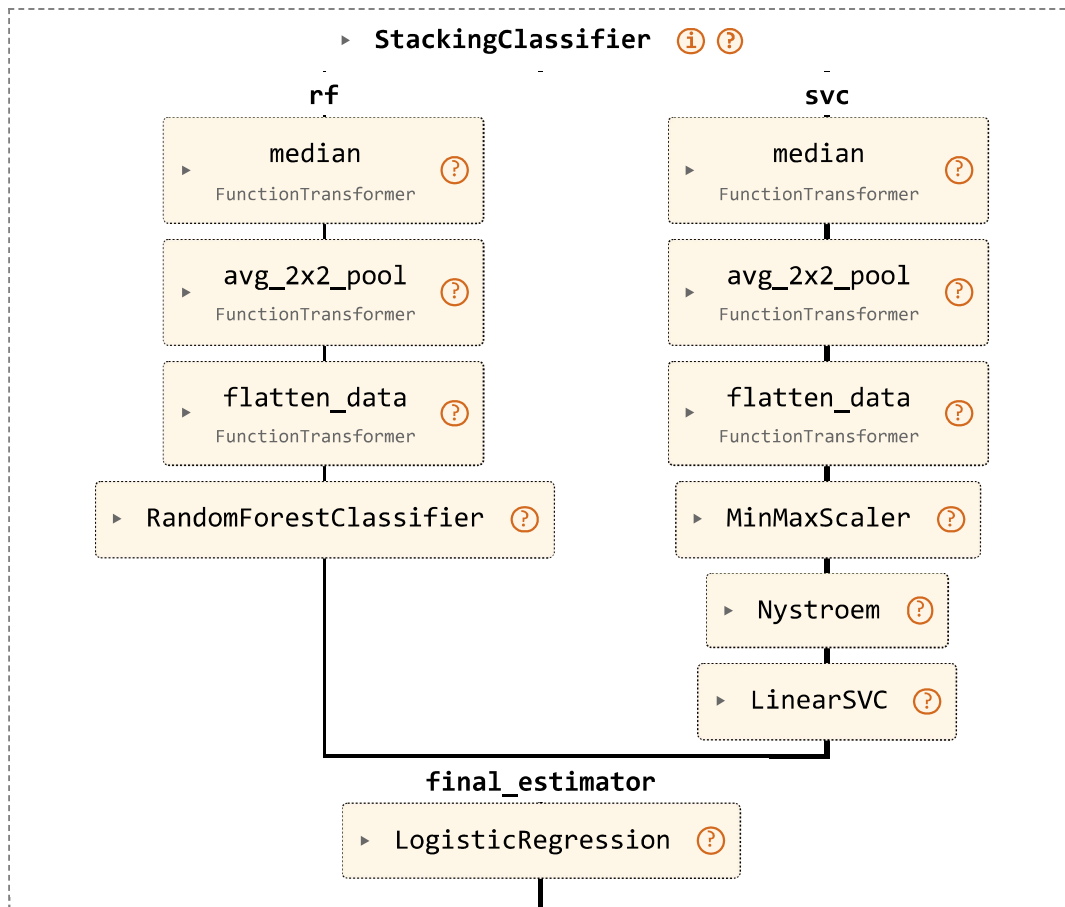
```
[Parallel(n_jobs=1)]: Done    2 out of   2 | elapsed:   14.5s finished
```

```
[Parallel(n_jobs=1)]: Done    2 out of   2 | elapsed:    5.8s finished
```

```
Best stacking params: {'final_estimator__C': np.float64(8.706020878304859), 'svc__kernel__n_components': 500, 'svc__svc__C': np.float64(0.26587543983272705)}
```

```
Best CV accuracy: 0.7059
```

Out[11]:



Note: n_iter and cv were reduced to decrease training speed. Likely at the cost of performance.

Model 2: Soft Voting Classifier

```

In [12]: from models import build_voting_classifier

voting_param_dist = {
    "rf_model_n_estimators": randint(100, 400),
    "rf_model_max_features": [0.2, 0.25, 0.33, "sqrt"],
    "rf_model_min_samples_leaf": randint(1, 8),
    "rf_model_max_depth": [None, 20, 40],
    "ada_model_n_estimators": randint(100, 400),
    "ada_model_learning_rate": loguniform(0.5, 2.0),
}

voting_search = RandomizedSearchCV(
    estimator=build_voting_classifier(),
    param_distributions=voting_param_dist,
    n_iter=8,
    cv=2,
    scoring="accuracy",
    n_jobs=-1,
    verbose=2,
    random_state=42,
    refit=True,
)

```

```
voting_search.fit(x_tr, y_tr)

print("Best voting params:", voting_search.best_params_)
print(f"Best CV accuracy: {voting_search.best_score_:.4f}")
voting_clf_tuned = voting_search.best_estimator_
```

Fitting 2 folds for each of 8 candidates, totalling 16 fits

Best voting params: {'ada__model__learning_rate': np.float64(0.6095150021599275), 'ada__model__n_estimators': 230, 'rf__model__max_depth': 20, 'rf__model__max_features': 0.2, 'rf__model__min_samples_leaf': 2, 'rf__model__n_estimators': 393}

Best CV accuracy: 0.6955

Model 3: SVD Soft Voting

```
In [13]: from models import build_svd_soft_voting

svd_param_dist = {
    "svd__n_components": [75, 100, 150, 200],
    "model__sgd__alpha": loguniform(1e-5, 1e-2),
    "model__sgd__penalty": ["l2", "elasticnet"],
    "model__svc__C": loguniform(0.1, 10),
    "model__svc__gamma": ["scale", "auto"],
}

svd_search = RandomizedSearchCV(
    estimator=build_svd_soft_voting(),
    param_distributions=svd_param_dist,
    n_iter=10,
    cv=2,
    scoring="accuracy",
    n_jobs=-1,
    verbose=2,
    random_state=42,
    refit=True,
)

svd_search.fit(x_tr, y_tr)

print("Best SVD soft-voting params:", svd_search.best_params_)
print(f"Best CV accuracy: {svd_search.best_score_:.4f}")
svd_clf_tuned = svd_search.best_estimator_
```

Fitting 2 folds for each of 10 candidates, totalling 20 fits

Best SVD soft-voting params: {'model__sgd__alpha': np.float64(0.0002246455168053261), 'model__sgd__penalty': 'elasticnet', 'model__svc__C': np.float64(7.902619549708233), 'model__svc__gamma': 'auto', 'svd__n_components': 100}

Best CV accuracy: 0.6506

Model 4: Wide & Deep

```
In [18]: from functools import partial
from sklearn.decomposition import PCA
from sklearn.preprocessing import MinMaxScaler
from models import build_tuned_wide_deep
```

```

x_va_flat = x_va.reshape(x_va.shape[0], -1).astype(np.float32)

scaler_wd = MinMaxScaler()
x_tr_s = scaler_wd.fit_transform(x_tr_flat)
x_va_s = scaler_wd.transform(x_va_flat)

thresholds = ["0.90", "0.95", "0.99", "0.999"]
n_by_threshold = {}
for t in thresholds:
    p = PCA(n_components=float(t), random_state=42).fit(x_tr_s)
    n_by_threshold[t] = p.n_components_

n_max = n_by_threshold["0.999"]

pca_full = PCA(n_components=n_max, random_state=42).fit(x_tr_s)
x_tr_pca = pca_full.transform(x_tr_s).astype(np.float32)
x_va_pca = pca_full.transform(x_va_s).astype(np.float32)

norm_raw = tf.keras.layers.Normalization()
norm_raw.adapt(x_tr_flat.astype(np.float32))
mean, var, _ = norm_raw.get_weights()

x_tr_wd = np.concatenate([x_tr_pca, x_tr_flat.astype(np.float32)], axis=1)
x_va_wd = np.concatenate([x_va_pca, x_va_flat.astype(np.float32)], axis=1)

```

To explain: The way the wide and deep network was implemented was as a KerasClassifier shown to us in the TextBook. Because of this PCA was implemented as part of a pipeline as you can see in models.py. This became challenging once it came time to tune though. I wanted the flexibility of architecture screening that Keras Tuning provided but in the current implementation of Wide And Deep that was impossible.

Above is the solution. Basically all Preprocessing steps are done ahead of time. PCA thresholds are calculated ahead of time. Then for the maximum number of components PCA is performed

Normalization mean and variance are extracted ahead of time as well.

The threshold dictionary, maximum number of PCA components, mean, and variance are then all passed to the model at build time

depending on the threshold the tuner selects only that number of PCA components is fed to the Wide Network. All of the regular inputs are always fed to the deep network. This is why we included n_max so we always know what that cutoff is. Finally the mean and variance are used in the normalization layer of the model.

For actual code implementation please see the attached models.py file

finally a "partial function" is used so we can actually pass these hyperparameters to the tuner.

This was taken from this stack overflow:

<https://stackoverflow.com/questions/69790356/how-to-pass-fix-hyperparameters-as-variables-for-keras-tuner>

```
In [20]: import keras_tuner as kt

wd_tuner = kt.Hyperband(
    hypermodel=partial(
        build_tuned_wide_deep,
        n_by_threshold=n_by_threshold,
        n_max=n_max,
        mean=mean,
        variance=var,
    ),
    objective="val_accuracy",
    max_epochs=20,
    factor=3,
    hyperband_iterations=1,
    directory="./wd_tuning",
    project_name="wd_search",
    overwrite=True,
    seed=42,
)

wd_tuner.search(
    x_tr_wd, y_tr,
    validation_data=(x_va_wd, y_va),
    sample_weight=sample_weights,
    batch_size=128,
    callbacks=[early_stopping_cb],
    verbose=1,
)
```

Trial 30 Complete [00h 00m 17s]
val_accuracy: 0.7285000085830688

Best val_accuracy So Far: 0.7379999756813049
Total elapsed time: 00h 03m 43s

Batch size of 128 was selected for speed.

```
In [28]: best_wd_hp = wd_tuner.get_best_hyperparameters(num_trials=1)[0]
best_wd_model = wd_tuner.hypermodel.build(best_wd_hp)
best_wd_model.summary()
print(wd_tuner.results_summary(1))
```

Model: "functional_7"

Layer (type)	Output Shape	Param #	Connected to
input_layer_7 (InputLayer)	(None, 1097)	0	-
get_item_15 (GetItem)	(None, 784)	0	input_layer_7[0]...
normalization_7 (Normalization)	(None, 784)	0	get_item_15[0][0]
dense_22 (Dense)	(None, 512)	401,920	normalization_7[...
dropout_12 (Dropout)	(None, 512)	0	dense_22[0][0]
dense_23 (Dense)	(None, 512)	262,656	dropout_12[0][0]
get_item_14 (GetItem)	(None, 21)	0	input_layer_7[0]...
dropout_13 (Dropout)	(None, 512)	0	dense_23[0][0]
concatenate_7 (Concatenate)	(None, 533)	0	get_item_14[0][0]... dropout_13[0][0]
dense_24 (Dense)	(None, 4)	2,136	concatenate_7[0]...

Total params: 666,712 (2.54 MB)

Trainable params: 666,712 (2.54 MB)

Non-trainable params: 0 (0.00 B)

Results summary
 Results in ./wd_tuning\wd_search
 Showing 1 best trials
 Objective(name="val_accuracy", direction="max")

Trial 0016 summary
 Hyperparameters:
 pca_threshold: 0.90
 activation: relu
 n_deep_layers: 2
 use_batch_norm: False
 dropout_rate: 0.1
 learning_rate: 0.00019785289659117255
 deep_units_0: 512
 deep_units_1: 512
 tuner/epochs: 20
 tuner/initial_epoch: 7
 tuner/bracket: 2
 tuner/round: 2
 tuner/trial_id: 0014
 deep_units_2: 192
 Score: 0.7379999756813049
 None

```
In [23]: from models import preprocess_for_cnn

x_tr_cnn = preprocess_for_cnn(x_tr)
x_va_cnn = preprocess_for_cnn(x_va)
```

Preprocess for CNN is a simple wrapper function that both median filters and adds the necessary channel

Model 5: CNN

```
In [29]: import keras_tuner as kt
from models import build_tuned_cnn

cnn_tuner = kt.Hyperband(
    hypermodel=build_tuned_cnn,
    objective="val_accuracy",
    max_epochs=20,
    factor=3,
    hyperband_iterations=1,
    directory="./cnn_tuning",
    project_name="cnn_search",
    overwrite=True,
    seed=42,
)

cnn_tuner.search(
    x_tr_cnn, y_tr,
    validation_data=(x_va_cnn, y_va),
    sample_weight=sample_weights,
    batch_size=128,
    callbacks=[early_stopping_cb],
```

```
verbose=1,
)
```

Trial 30 Complete [00h 06m 43s]
val_accuracy: 0.8220000267028809

Best val_accuracy So Far: 0.8220000267028809
Total elapsed time: 00h 48m 59s

```
In [30]: best_cnn_hp = cnn_tuner.get_best_hyperparameters(num_trials=1)[0]
best_cnn_model = cnn_tuner.hypermodel.build(best_cnn_hp)
best_cnn_model.summary()

print(cnn_tuner.results_summary(1))
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0
conv2d_3 (Conv2D)	(None, 28, 28, 64)	640
batch_normalization_3 (BatchNormalization)	(None, 28, 28, 64)	256
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_4 (Conv2D)	(None, 14, 14, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 14, 14, 128)	512
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_5 (Conv2D)	(None, 7, 7, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 7, 7, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten_1 (Flatten)	(None, 1152)	0
dense_1 (Dense)	(None, 4)	4,612

Total params: 227,972 (890.52 KB)

Trainable params: 227,332 (888.02 KB)

Non-trainable params: 640 (2.50 KB)

Results summary
 Results in ./cnn_tuning\cnn_search
 Showing 1 best trials
 Objective(name="val_accuracy", direction="max")

Trial 0029 summary
 Hyperparameters:
 activation: relu
 n_conv_blocks: 3
 kernel_size: 3
 use_batch_norm: True
 dropout_rate: 0.0
 n_dense_layers: 0
 learning_rate: 0.00330588674286921
 filters_0: 64
 filters_1: 128
 filters_2: 128
 dense_units_0: 64
 tuner/epochs: 20
 tuner/initial_epoch: 0
 tuner/bracket: 0
 tuner/round: 0
 Score: 0.8220000267028809
 None

ResNet — Hyperband Tuning

```
In [31]: from models import prep_for_cnn

x_tr_res = prep_for_cnn(x_tr)
x_va_res = prep_for_cnn(x_va)
```

No median Filtering done here

```
In [32]: from models import build_tuned_resnet

resnet_tuner = kt.Hyperband(
    hypermodel=build_tuned_resnet,
    objective="val_accuracy",
    max_epochs=20,
    factor=3,
    hyperband_iterations=1,
    directory="./resnet_tuning",
    project_name="resnet_search",
    overwrite=True,
    seed=42,
)

resnet_tuner.search(
    x_tr_res, y_tr,
    validation_data=(x_va_res, y_va),
    sample_weight=sample_weights,
    batch_size=128,
    callbacks=[early_stopping_cb],
```

```
verbose=1,
)
```

Trial 30 Complete [00h 17m 29s]
val_accuracy: 0.9004999995231628

Best val_accuracy So Far: 0.902999997138977
Total elapsed time: 04h 23m 51s

```
In [33]: best_resnet_hp      = resnet_tuner.get_best_hyperparameters(num_trials=1)[0]
best_resnet_model = resnet_tuner.hypermodel.build(best_resnet_hp)
best_resnet_model.summary()
print(resnet_tuner.results_summary(1))
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0
conv2d_15 (Conv2D)	(None, 28, 28, 64)	576
batch_normalization_15 (BatchNormalization)	(None, 28, 28, 64)	256
activation_1 (Activation)	(None, 28, 28, 64)	0
residual_unit_6 (ResidualUnit)	(None, 28, 28, 64)	74,240
residual_unit_7 (ResidualUnit)	(None, 28, 28, 64)	74,240
residual_unit_8 (ResidualUnit)	(None, 14, 14, 128)	230,912
residual_unit_9 (ResidualUnit)	(None, 14, 14, 128)	295,936
residual_unit_10 (ResidualUnit)	(None, 7, 7, 256)	920,576
residual_unit_11 (ResidualUnit)	(None, 7, 7, 256)	1,181,696
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 256)	0
dense_1 (Dense)	(None, 64)	16,448
dense_2 (Dense)	(None, 4)	260

Total params: 2,795,140 (10.66 MB)

Trainable params: 2,790,660 (10.65 MB)

Non-trainable params: 4,480 (17.50 KB)

Results summary
 Results in ./resnet_tuning/resnet_search
 Showing 1 best trials
 Objective(name="val_accuracy", direction="max")

Trial 0025 summary

Hyperparameters:

activation: relu

n_stages: 3

base_filters: 64

units_per_stage: 2

dropout_rate: 0.0

use_dense: True

learning_rate: 0.0008019212661756024

dense_units: 64

tuner/epochs: 20

tuner/initial_epoch: 7

tuner/bracket: 1

tuner/round: 1

tuner/trial_id: 0023

Score: 0.902999997138977

None

Final Evaluation on Test Set

```
In [34]: import matplotlib.pyplot as plt
import numpy as np
import time
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, f1_score, ConfusionMat
from sklearn.preprocessing import label_binarize
from sklearn.utils.class_weight import compute_sample_weight

class_names = ["Normal", "CNV", "DME", "Drusen"]

sample_weights_full = compute_sample_weight(class_weight="balanced", y=train_labels

all_results = []

def evaluate_classifier(name, y_true, y_pred, y_proba, time):
    """
    Evaluates model for us, plots a confusion matrix, and ROC curve for each
    """
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, average="weighted", zero_division=1)
    f1 = f1_score(y_true, y_pred, average="weighted", zero_division=1)

    print(f"{name}")
    print(f"Accuracy: {acc:.4f}")
    print(f"Weighted Precision: {prec:.4f}")
    print(f"Weighted F1: {f1:.4f}")
    print(f"Training Time: {time:.4f}s")

    fig, ax = plt.subplots(1, 2, figsize=(14, 5))
```

```

ConfusionMatrixDisplay.from_predictions(
    y_true, y_pred,
    display_labels=class_names,
    ax=ax[0],
    colorbar=False,
    cmap="plasma",
)
ax[0].set_title(f"{name} - Confusion Matrix")
ax[0].tick_params(axis="x", rotation=15)

y_bin = label_binarize(y_true, classes=[0, 1, 2, 3])
colors = ["r", "b", "g", "y"]
aucs = []
for i, (cname, col) in enumerate(zip(class_names, colors)):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_proba[:, i])
    roc_auc = auc(fpr, tpr)
    aucs.append(roc_auc)
    ax[1].plot(fpr, tpr, color=col, lw=1.8,
               label=f"{cname} (AUC={roc_auc:.3f})")
ax[1].plot([0, 1], [0, 1], "k--", lw=1)
ax[1].set_xlabel("False Positive Rate")
ax[1].set_ylabel("True Positive Rate")
ax[1].set_title(f"{name} - ROC Curves (OvR)")
ax[1].legend(loc="lower right")

mean_auc = np.mean(aucs)

plt.tight_layout()
plt.show()
return {"name": name, "acc": acc, "prec": prec, "f1": f1, "time": time, "auc":

```

To quickly explain what the above function does (it's fairly straight forward though).

After the model is trained the accuracy (weighted recall), weighted precision and f1 score are all calculated and printed. A confusion matrix is calculated and displayed then does the same with AUC curves per class finally it calculates the Mean AUC and returns a dictionary of all results including training time (passed along to function ahead of time)

Model 1: Preprocessed Stacking

```

In [35]: start = time.time()
stacking_clf_tuned.fit(train_images, train_labels)
end = time.time()

t = end - start
stacking_pred = stacking_clf_tuned.predict(test_images)
stacking_proba = stacking_clf_tuned.predict_proba(test_images)
all_results.append(
    evaluate_classifier("Preprocessed Stacking", test_labels, stacking_pred, stacki
)

```

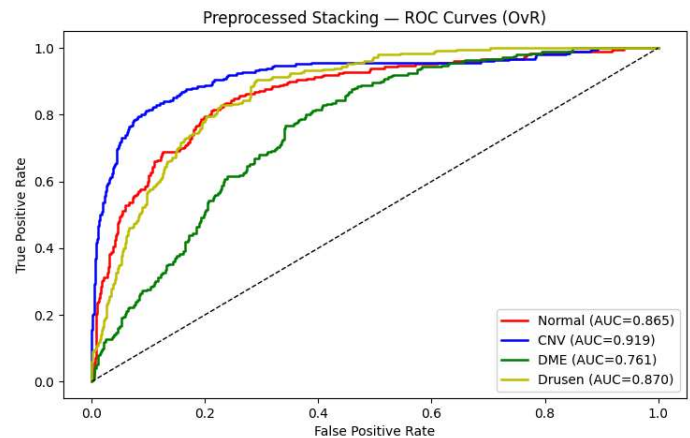
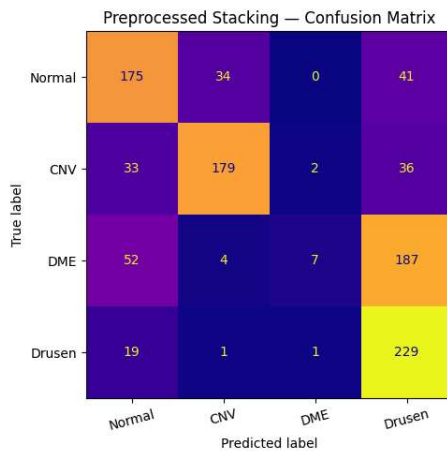
```

[Parallel(n_jobs=1)]: Done    2 out of   2 | elapsed: 17.0min finished
[Parallel(n_jobs=1)]: Done    2 out of   2 | elapsed: 16.4min finished

```

Preprocessed Stacking

Accuracy: 0.5900
 Weighted Precision: 0.6532
 Weighted F1: 0.5242
 Training Time: 3087.0103s



Model 2: Soft Voting Classifier

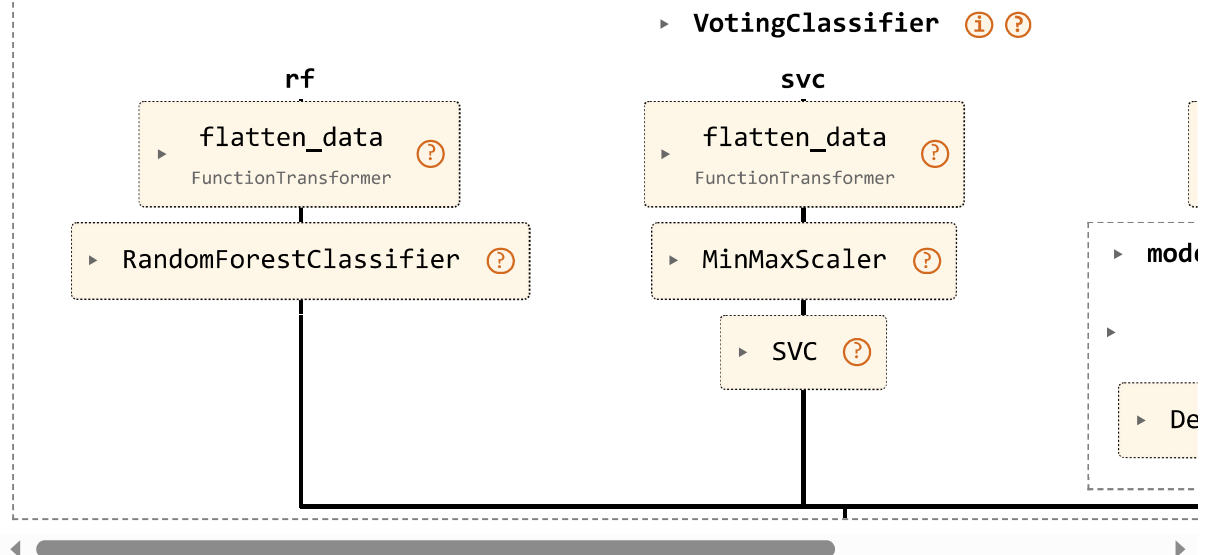
```
In [43]: best_params = voting_search.best_params_.copy()

best_params["verbose"] = 2
best_params["n_jobs"] = -1

voting_clf_final = build_voting_classifier()
voting_clf_final.set_params(**best_params)

voting_clf_final #n_jobs and verbosity wasn't on for Stacking
```

Out[43]:



```
In [ ]: #dump models
import joblib

joblib.dump(voting_clf_final, 'voting_clf.joblib')
joblib.dump(svd_clf_tuned, )
```

```
In [44]: start = time.time()
voting_clf_final.fit(train_images, train_labels)
end = time.time()

t = end - start
voting_pred = voting_clf_tuned.predict(test_images)
voting_proba = voting_clf_tuned.predict_proba(test_images)
all_results.append(
    evaluate_classifier("Soft Voting", test_labels, voting_pred, voting_proba, time
    )
```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[44], line 2
      1 start = time.time()
----> 2 voting_clf_final.fit(train_images, train_labels)
      3 end = time.time()
      4
      5 t = end - start

File c:\Users\Owner\anaconda3\envs\MLEnvironment\Lib\site-packages\sklearn\base.py:1
336, in _fit_context.<locals>.decorator.<locals>.wrapper(estimator, *args, **kwargs)
    1329 estimator._validate_params()
    1331 with config_context(
    1332     skip_parameter_validation=(
    1333         prefer_skip_nested_validation or global_skip_validation
    1334     )
    1335 ):
-> 1336     return fit_method(estimator, *args, **kwargs)

File c:\Users\Owner\anaconda3\envs\MLEnvironment\Lib\site-packages\sklearn\ensemble
\_voting.py:405, in VotingClassifier.fit(self, X, y, **fit_params)
    402 self.classes_ = self.le_.classes_
    403 transformed_y = self.le_.transform(y)
--> 405 return super().fit(X, transformed_y, **fit_params)

File c:\Users\Owner\anaconda3\envs\MLEnvironment\Lib\site-packages\sklearn\ensemble
\_voting.py:99, in _BaseVoting.fit(self, X, y, **fit_params)
     94     if "sample_weight" in fit_params:
     95         routed_params[name].fit["sample_weight"] = fit_params[
     96             "sample_weight"
     97         ]
--> 99 self.estimators_ = Parallel(n_jobs=self.n_jobs)(
    100     delayed(_fit_single_estimator)(
    101         clone(clf),
    102         X,
    103         y,
    104         fit_params=routed_params[name]["fit"],
    105         message_clsname="Voting",
    106         message=self._log_message(name, idx + 1, len(clfs)),
    107     )
    108     for idx, (name, clf) in enumerate(zip(names, clfs))
    109     if clf != "drop"
    110 )
    112 self.named_estimators_ = Bunch()
    114 # Uses 'drop' as placeholder for dropped estimators

File c:\Users\Owner\anaconda3\envs\MLEnvironment\Lib\site-packages\sklearn\utils\par
allel.py:91, in Parallel.__call__(self, iterable)
     79 warning_filters = (
     80     filters_func() if filters_func is not None else warnings.filters
     81 )
     83 iterable_with_config_and_warning_filters = (
     84     (
     85         _with_config_and_warning_filters(delayed_func, config, warning_filt
rs),
     (...))
     89     for delayed_func, args, kwargs in iterable

```

```

90 )
--> 91 return super().__call__(iterable_with_config_and_warning_filters)

File c:\Users\Owner\anaconda3\envs\MLEnvironment\Lib\site-packages\joblib\parallel.p
y:2072, in Parallel.__call__(self, iterable)
2066 # The first item from the output is blank, but it makes the interpreter
2067 # progress until it enters the Try/Except block of the generator and
2068 # reaches the first `yield` statement. This starts the asynchronous
2069 # dispatch of the tasks to the workers.
2070 next(output)
-> 2072 return output if self.return_generator else list(output)

File c:\Users\Owner\anaconda3\envs\MLEnvironment\Lib\site-packages\joblib\parallel.p
y:1682, in Parallel._get_outputs(self, iterator, pre_dispatch)
1679     yield
1681     with self._backend.retrieval_context():
-> 1682         yield from self._retrieve()
1684 except GeneratorExit:
1685     # The generator has been garbage collected before being fully
1686     # consumed. This aborts the remaining tasks if possible and warn
1687     # the user if necessary.
1688     self._exception = True

File c:\Users\Owner\anaconda3\envs\MLEnvironment\Lib\site-packages\joblib\parallel.p
y:1800, in Parallel._retrieve(self)
1789 if self.return_ordered:
1790     # Case ordered: wait for completion (or error) of the next job
1791     # that have been dispatched and not retrieved yet. If no job
(...) 1795     # control only have to be done on the amount of time the next
1796     # dispatched job is pending.
1797     if (nb_jobs == 0) or (
1798         self._jobs[0].get_status(timeout=self.timeout) == TASK_PENDING
1799     ):
-> 1800         time.sleep(0.01)
1801         continue
1803 elif nb_jobs == 0:
1804     # Case unordered: jobs are added to the list of jobs to
1805     # retrieve `self._jobs` only once completed or in error, which
(...) 1811     # timeouts before any other dispatched job has completed and
1812     # been added to `self._jobs` to be retrieved.

KeyboardInterrupt:

```

Model 3: SVD Soft Voting

```

In [ ]: start = time.time()
svd_clf_tuned.fit(train_images, train_labels)
end = time.time()

t = end - start
svd_pred = svd_clf_tuned.predict(test_images)
svd_proba = svd_clf_tuned.predict_proba(test_images)
all_results.append(
    evaluate_classifier("SVD Soft Voting", test_labels, svd_pred, svd_proba, time=t
)

```

Model 4: Wide & Deep

```

In [ ]: x_tr_flat_full = train_images.reshape(len(train_images), -1).astype(np.float32)
x_tr_wd_full = np.concatenate([
    pca_full.transform(scaler_wd.transform(x_tr_flat_full)).astype(np.float32),
    x_tr_flat_full,
], axis=1)

x_te_flat = test_images.reshape(len(test_images), -1).astype(np.float32)
x_te_wd = np.concatenate([
    pca_full.transform(scaler_wd.transform(x_te_flat)).astype(np.float32),
    x_te_flat,
], axis=1)

wd_final = wd_tuner.hypermodel.build(best_wd_hp)
start = time.time()
wd_final.fit(
    x_tr_wd_full, train_labels,
    epochs=100,
    batch_size=256,
    sample_weight=sample_weights_full,
    validation_split=0.1,
    callbacks=[early_stopping_cb],
    verbose=1,
)
end = time.time()

t = end - start
wd_proba = wd_final.predict(x_te_wd)
wd_pred = np.argmax(wd_proba, axis=1)
all_results.append(
    evaluate_classifier("Wide & Deep", test_labels, wd_pred, wd_proba, time=t)
)

```

Prepare Full-Dataset CNN Inputs

```

In [ ]: x_tr_cnn_full = preprocess_for_cnn(train_images)
x_te_cnn = preprocess_for_cnn(test_images)

```

Model 5: CNN

```

In [ ]: cnn_final = cnn_tuner.hypermodel.build(best_cnn_hp)
start = time.time()
cnn_final.fit(
    x_tr_cnn_full, train_labels,
    epochs=100,
    batch_size=256,
    sample_weight=sample_weights_full,
    validation_split=0.1,
    callbacks=[early_stopping_cb],
    verbose=1,
)
end = time.time()

```

```

t = end - start
cnn_proba = cnn_final.predict(x_te_cnn)
cnn_pred = np.argmax(cnn_proba, axis=1)
all_results.append(
    evaluate_classifier("CNN", test_labels, cnn_pred, cnn_proba, time=t)
)

```

Model 6: Resnet

```

In [ ]: x_tr_res_full = prep_for_cnn(train_images)
        x_te_res_full = prep_for_cnn(test_images)

```

```

In [ ]: resnet_final = resnet_tuner.hypermodel.build(best_resnet_hp)
        start = time.time()
        resnet_final.fit(
            x_tr_res_full, train_labels,
            epochs=100,
            batch_size=128,
            sample_weight=sample_weights_full,
            validation_split=0.1,
            callbacks=[early_stopping_cb],
            verbose=1,
        )
        end = time.time()

        t = end - start
        resnet_proba = resnet_final.predict(x_te_res_full)
        resnet_pred = np.argmax(resnet_proba, axis=1)
        all_results.append(
            evaluate_classifier("ResNet", test_labels, resnet_pred, resnet_proba, time=t)
        )

```

Final Summary

```

In [ ]: col_names = {
        "acc": "Accuracy",
        "prec": "Weighted Precision",
        "f1": "Weighted F1",
        "time": "Train Time (s)",
        "auc": "Mean AUC",
    }

    summary_df = (
        pd.DataFrame(all_results)
        .set_index("name")
        .rename(columns=col_names)
        .sort_values("Weighted F1", ascending=False)
    )

    print(summary_df.to_string(float_format="{:.4f}".format))

```